Visión Artificial, proyecto final
Material de apoyo

Paso a paso

Una ruta sugerida para cada uno de los 8 proyectos, desde los datos hasta la base de datos y el reporte para el cliente.

Uso	Proyectos	Docente	Contacto
Guía orientativa	8 rutas	Eduardo Sierra Fragozo	elsierra@uniguajira.edu.co

Una ruta, no una receta

Cada ruta propone un camino que funciona. Pueden cambiar herramientas, modelos o pasos si tienen una buena razón, y explicarla en el informe es parte del análisis técnico.

1 **Mismo esqueleto para todos**

Datos, modelo, lógica del problema, base de datos, interfaz o reporte, métricas. Las semanas marcadas en cada paso son una referencia.

2 **Base de datos desde el principio**

Creen las tablas en la semana 1 aunque estén vacías. SQLite basta para empezar y se puede migrar a PostgreSQL si hacen despliegue.

3 **Todo pensado para el cliente**

Cada paso debe acercarlos al indicador de éxito que definieron en la ficha del cliente.

Estructura sugerida del repositorio

```
proyecto/
  data/            imágenes, videos y etiquetas (o enlace)
  notebooks/       exploración y entrenamiento
  src/
    detector.py    modelo y lógica de visión
    db.py          conexión y consultas a la base de datos
    app.py         interfaz o API, si la hacen
  db/schema.sql    creación de tablas
  models/          pesos entrenados (o enlace)
  README.md        cómo instalar y ejecutar
```

1 Aforo vehicular con estimación de velocidad

Para una Secretaría de Tránsito, una concesión vial o una empresa de transporte

Video fijo → YOLO detecta → ByteTrack asigna ID → Línea virtual cuenta → Velocidad en km/h → Base de datos → Reporte por hora

1. **Graben el video.** Desde un punto alto y fijo, entre 10 y 15 minutos. Midan en la calle la distancia real entre dos marcas, por ejemplo dos postes a 10 m. Semana 1
2. **Detecten y sigan.** Usen YOLO preentrenado en COCO (carro, moto, bus, camión) con `model.track(source, persist=True, tracker="bytetrack.yaml")`. Cada vehículo recibe un ID estable. Semana 1
3. **Cuenten con una línea virtual.** Guarden el centroide de cada ID y cuenten cuando cruza la línea, detectando el cambio de signo del producto cruz. Cada ID se cuenta una sola vez. Semana 2
4. **Calculen la velocidad.** Con dos líneas separadas por la distancia real d, tiempo igual a cuadros entre cruces dividido por los fps, y velocidad igual a d sobre t, por 3.6 para km/h. Semana 2
5. **Guarden cada cruce.** Un registro por vehículo con tipo, hora y velocidad en la base de datos. Semana 2
6. **Validen.** Cuenten a mano 2 minutos de video y comparen. Si pueden, pasen un carro a velocidad conocida. Semana 3
7. **Reporte para el cliente.** Consultas SQL de vehículos por hora, por tipo y velocidad promedio. Semana 3

Tablas sugeridas

```
CREATE TABLE cruces (
  id INTEGER PRIMARY KEY,
  track_id INTEGER,
  tipo TEXT,
  momento TIMESTAMP,
  velocidad_kmh REAL,
  camara TEXT
);
```

El curso aquí

- Convoluciones dentro de YOLO
- Modelo de cámara para pasar de píxeles a metros
- Perspectiva, la escala cambia a lo largo de la vía
- Restas de posiciones y divisiones para la velocidad

Errores comunes

- Contar dos veces el mismo vehículo
- Leer mal los fps del video
- Usar una escala única cuando la perspectiva la deforma

Funciones y herramientas clave

model.track · cv2.VideoCapture · CAP_PROP_FPS · cv2.line · cv2.getPerspectiveTransform · sqlite3

2 Medidor de objetos sin regla

Para una empresa de logística, una fábrica que despacha cajas o una carpintería

Cámara cenital → Calibración → Corrección de distorsión → Segmentación → Contornos → Escala px a cm → Base de datos

1. **Calibren.** Impriman un tablero de ajedrez de 9 por 6 con cuadros de tamaño conocido. Tomen 15 a 20 fotos en distintas posiciones y usen `findChessboardCorners`, `cornerSubPix` y `calibrateCamera`. Guarden K y la distorsión en un `.npz`. Semana 1
2. **Corrijan la distorsión.** Apliquen `cv2.undistort` a cada imagen antes de medir. Semana 1
3. **Fijen la escala.** Usen el tablero o un marcador ArUco sobre la mesa para saber cuántos milímetros mide un píxel en ese plano. Semana 1
4. **Segmenten.** Escala de grises, `GaussianBlur`, umbral de Otsu o Canny, cierre morfológico y `findContours`. Semana 2
5. **Midan.** Con `minAreaRect` obtengan ancho y alto en píxeles, multipliquen por la escala y dibujen la medida sobre la imagen. Semana 2
6. **Validen.** Midan 10 objetos con regla y armen una tabla con error promedio y error máximo. Semana 3
7. **Guarden.** Cada medición con su foto en la base de datos, para que el cliente consulte el historial. Semana 3

Tablas sugeridas

```
CREATE TABLE mediciones (
  id INTEGER PRIMARY KEY,
  objeto TEXT,
  ancho_cm REAL,
  alto_cm REAL,
  momento TIMESTAMP,
  ruta_imagen TEXT
);
```

El curso aquí

- Modelo de cámara, matriz intrínseca y distorsión
- Filtros de suavizado
- Umbralización y morfología
- Homografía si la cámara está inclinada

Errores comunes

- Cambiar el zoom o el foco después de calibrar
- Sombras que rompen el umbral, usen un fondo uniforme
- Medir sin corregir la distorsión

Funciones y herramientas clave

cv2.calibrateCamera · cv2.undistort · cv2.aruco · cv2.threshold · cv2.findContours · cv2.minAreaRect

3 Detector de elementos de protección personal

Para una constructora con obras en la ciudad o una empresa de seguridad industrial

Fotos de obra → Etiquetado → YOLO entrenado → Asociación persona y EPP → Zona restringida → Alerta → Base de datos

1 **Armen el dataset.** Al menos 300 fotos, propias más un dataset público de cascos. Etiqueten persona, casco y chaleco en Roboflow y exporten en formato YOLO, dividido 70, 20 y 10. Semana 1

2 **Entrenen.** YOLO pequeño en Colab, entre 50 y 100 épocas con imágenes de 640. Revisen mAP50 y la matriz de confusión. Semana 2

3 **Asocien el equipo.** Un casco pertenece a una persona si su centro cae en el tercio superior de la caja de esa persona. El chaleco, en la zona del torso. Semana 2

4 **Definan la zona.** Un polígono y `cv2.pointPolygonTest` sobre el punto de los pies de cada persona. Semana 2

5 **Alerta estable.** Solo alertar si la falta se mantiene varios cuadros seguidos, para evitar parpadeos. Semana 2

6 **Guarden eventos.** Cada incumplimiento con zona, hora, qué falta y captura en la base de datos. Semana 3

7 **Midan.** Precisión y recall por clase en test, y casos de falla como gorras confundidas con cascos. Semana 3

Tablas sugeridas

```
CREATE TABLE eventos (
  id INTEGER PRIMARY KEY,
  momento TIMESTAMP,
  zona TEXT,
  falta_casco BOOLEAN,
  falta_chaleco BOOLEAN,
  confianza REAL,
  ruta_captura TEXT
);
```

El curso aquí

- CNN y YOLO entrenados con datos propios
- IoU, intersección sobre unión
- Umbrales de confianza
- Iluminación y distancia de la cámara

Errores comunes

- Pocas fotos de personas sin casco
- Mismo sitio y misma luz en train y test
- Alertar por cada cuadro y saturar al cliente

Funciones y herramientas clave

Roboflow · model.train · model.val · cv2.pointPolygonTest · cv2.polylines · sqlite3

4 Lector de placas vehiculares

Para un conjunto residencial, un centro comercial o un parqueadero público

Cámara de acceso → YOLO detecta placa → Recorte → Preprocesamiento → OCR → Validación de formato → Base de datos

1. **Dataset de placas.** Fotos propias en un parqueadero más datasets públicos. Etiqueten la clase placa y entrenen YOLO. Semana 1
2. **Recorten con margen.** Agranden el recorte 2 o 3 veces para que el OCR lea mejor. Semana 1
3. **Preprocesen.** Gris, `bilateralFilter`, umbral de Otsu o adaptativo y morfología. Prueben varias combinaciones. Semana 2
4. **Lean.** EasyOCR con `allowlist` de letras mayúsculas y números. Semana 2
5. **Validen el formato.** Carros con tres letras y tres números, motos con tres letras, dos números y una letra. Corrijan O y 0, I y 1 según la posición. Semana 2
6. **Voten.** Lean la misma placa en varios cuadros y quédense con la lectura más repetida. Semana 3
7. **Registren accesos.** Entradas y salidas en la base de datos, con consulta de tiempo de permanencia. Semana 3

Tablas sugeridas

```
CREATE TABLE vehiculos (
  placa TEXT PRIMARY KEY,
  tipo TEXT
);
CREATE TABLE accesos (
  id INTEGER PRIMARY KEY,
  placa TEXT REFERENCES
vehiculos,
  momento TIMESTAMP,
  sentido TEXT,
  confianza REAL
);
```

El curso aquí

- Filtros y umbralización
- Operaciones morfológicas
- CNN dentro de YOLO y del OCR
- Perspectiva, placas inclinadas

Errores comunes

- Guardar lecturas sin validar
- Placas inclinadas sin corregir
- Reflejos de noche

Funciones y herramientas clave

model.predict · cv2.bilateralFilter · cv2.adaptiveThreshold · cv2.morphologyEx · easyocr.Reader · re.match

5 Detector de somnolencia del conductor

Para una empresa de transporte intermunicipal o de carga

Webcam → Malla facial → EAR y MAR → Umbral en el tiempo → Alarma → Base de datos

1. **Malla facial.** MediaPipe Face Landmarker para obtener 6 puntos por ojo y los puntos de la boca. Semana 1
2. **Calculen EAR.** La suma de las dos distancias verticales del ojo, dividida por dos veces la distancia horizontal. Hagan lo mismo para la boca. Semana 1
3. **Calibren por persona.** Durante 5 segundos con ojos abiertos midan el EAR medio y usen el 75 % como umbral. Semana 2
4. **Decidan en el tiempo.** Somnolencia si el EAR está bajo el umbral más de 1.5 s, bostezo si la boca supera su umbral más de 2 s. Conviertan segundos a cuadros con los fps. Semana 2
5. **Alarma.** Sonido y aviso en pantalla. Semana 2
6. **Guarden sesiones y eventos.** Conductor, inicio, fin y cada evento con su duración. Semana 3
7. **Prueben.** 5 personas, luz normal y baja, con y sin gafas. Tabla de aciertos y falsas alarmas. Semana 3

Tablas sugeridas

```
CREATE TABLE sesiones (
  id INTEGER PRIMARY KEY,
  conductor TEXT,
  inicio TIMESTAMP,
  fin TIMESTAMP
);
CREATE TABLE eventos (
  id INTEGER PRIMARY KEY,
  sesion_id INTEGER
REFERENCES sesiones,
  momento TIMESTAMP,
  tipo TEXT,
  duracion_s REAL
);
```

El curso aquí

- Distancias euclidianas, aritmética pura
- Ecualización de histograma
- Redes convolucionales dentro de MediaPipe
- Iluminación y posición de la cámara

Errores comunes

- Un umbral fijo para todas las personas
- Confundir un parpadeo normal con sueño
- No probar con la cara de lado

Funciones y herramientas clave

mediapipe FaceLandmarker · numpy.linalg.norm · cv2.equalizeHist · pygame.mixer · sqlite3

6 Parqueadero inteligente

Para un centro comercial, una clínica o la propia universidad

Cámara fija → Mapa de cupos → Estado de cada cupo → Suavizado → Base de datos → Tablero

1. **Graben y mapeen.** Video propio o maqueta con carros de juguete. Marquen con clics el polígono de cada cupo y guárdenlo en JSON. Semana 1
2. **Opción A, detección.** YOLO detecta carros y un cupo está ocupado si la IoU entre carro y cupo supera 0.3. Semana 2
3. **Opción B, clasificación.** Recorten cada cupo y entrenen una CNN binaria, libre u ocupado, con transfer learning sobre MobileNetV2. Semana 2
4. **Suavicen.** El estado cambia solo si se mantiene varios cuadros, para evitar parpadeos. Semana 2
5. **Guarden cambios de estado.** Cupo, hora y si quedó libre u ocupado. Semana 2
6. **Tablero.** Streamlit con el mapa coloreado y el número de cupos libres. Semana 3
7. **Midan.** Exactitud por cupo en cuadros de prueba y casos con sombras o carros mal parqueados. Semana 3

Tablas sugeridas

```
CREATE TABLE cupos (
  id INTEGER PRIMARY KEY,
  zona TEXT,
  poligono_json TEXT
);
CREATE TABLE estados (
  id INTEGER PRIMARY KEY,
  cupo_id INTEGER
REFERENCES cupos,
  momento TIMESTAMP,
  ocupado BOOLEAN
);
```

El curso aquí

- CNN y YOLO
- IoU y máscaras
- Perspectiva de la cámara sobre los cupos
- Normalización de los recortes

Errores comunes

- Cupos dibujados sin tener en cuenta la perspectiva
- Sombras que parecen carros
- Entrenar con un solo momento del día

Funciones y herramientas clave

cv2.setMouseCallback · cv2.fillPoly · model.predict · tf.keras.applications.MobileNetV2 · streamlit

7 Clasificador de calidad de frutas

Para una comercializadora, un centro de acopio o un supermercado

Estación de fotos → Preprocesamiento → CNN con transfer learning → Clasificación → Base de datos → Reporte de lote

1. **Estación de fotos.** Una fruta local, fondo y luz constantes, al menos 200 fotos por clase, buena y dañada, o tres clases si quieren. Semana 1
2. **Dividan bien.** Separen train y test por fruta, no por foto, para que la misma fruta no quede en ambos. Semana 1
3. **Dos versiones.** A con imágenes originales y B con preprocesamiento, por ejemplo HSV, CLAHE o realce de manchas. Semana 2
4. **Transfer learning.** MobileNetV2 o ResNet50, primero congelada entrenando solo la cabeza, luego afinando las últimas capas, con data augmentation. Semana 2
5. **Evalúen.** Matriz de confusión, precisión y recall por clase, y comparación entre A y B. Semana 2
6. **Lotes.** Guarden cada fruta clasificada con su lote y calculen el porcentaje de rechazo por lote. Semana 3
7. **Reto.** Grad-CAM para ver en qué parte de la fruta se fija la red. Semana 3

Tablas sugeridas

```
CREATE TABLE lotes (
  id INTEGER PRIMARY KEY,
  proveedor TEXT,
  fecha DATE
);
CREATE TABLE frutas (
  id INTEGER PRIMARY KEY,
  lote_id INTEGER
REFERENCES lotes,
  clase TEXT,
  confianza REAL,
  ruta_imagen TEXT
);
```

El curso aquí

- Espacios de color y filtros
- CNN y transfer learning
- Normalización de entrada
- Iluminación constante, modelo de cámara

Errores comunes

- Fuga de datos entre train y test
- Clases desbalanceadas
- Que la red aprenda el fondo y no la fruta

Funciones y herramientas clave

cv2.cvtColor HSV · cv2.createCLAHE · ImageDataGenerator · MobileNetV2 · confusion_matrix · sqlite3

8 Detección de basura en playas de Riohacha

Para un hotel de playa, el operador de aseo de la ciudad o la Alcaldía

Fotos por tramo → Etiquetado → YOLO entrenado → Conteo → Base de datos por zona → Mapa de calor

1 **Dividan la playa.** Tramos de unos 100 m. Tomen fotos a una altura fija, anotando tramo, hora y ubicación. Semana 1

2 **Etiqueten.** Al menos 300 fotos en plástico, botella, lata y otros, complementando con el dataset público TACO. Semana 1

3 **Entrenen YOLO.** Los residuos son pequeños, así que prueben imágenes de 960 o dividir cada foto en partes. Semana 2

4 **Cuenten.** Residuos por foto y por clase, guardados con su tramo en la base de datos. Semana 2

5 **Mapa.** Consulta agregada por tramo y mapa de calor con folium, o gráfico de barras. Semana 3

6 **Midan.** Métricas por clase y falsos positivos con algas, conchas o huellas. Semana 3

7 **Informe al cliente.** Tramos prioritarios para limpieza y evolución si toman fotos en días distintos. Semana 3

Tablas sugeridas

```
CREATE TABLE tramos (
  id INTEGER PRIMARY KEY,
  nombre TEXT,
  lat REAL,
  lon REAL
);
CREATE TABLE detecciones (
  id INTEGER PRIMARY KEY,
  tramo_id INTEGER
REFERENCES tramos,
  momento TIMESTAMP,
  clase TEXT,
  confianza REAL
);
```

El curso aquí

- YOLO y CNN
- Resolución y distancia de la cámara
- Conteo y agregación
- Iluminación del sol y reflejos

Errores comunes

- Fotos a alturas distintas
- Pocas imágenes de objetos pequeños
- Confundir elementos naturales con basura

Funciones y herramientas clave

Roboflow · model.train imgsz=960 · SAHI · pandas groupby · folium.HeatMap · sqlite3